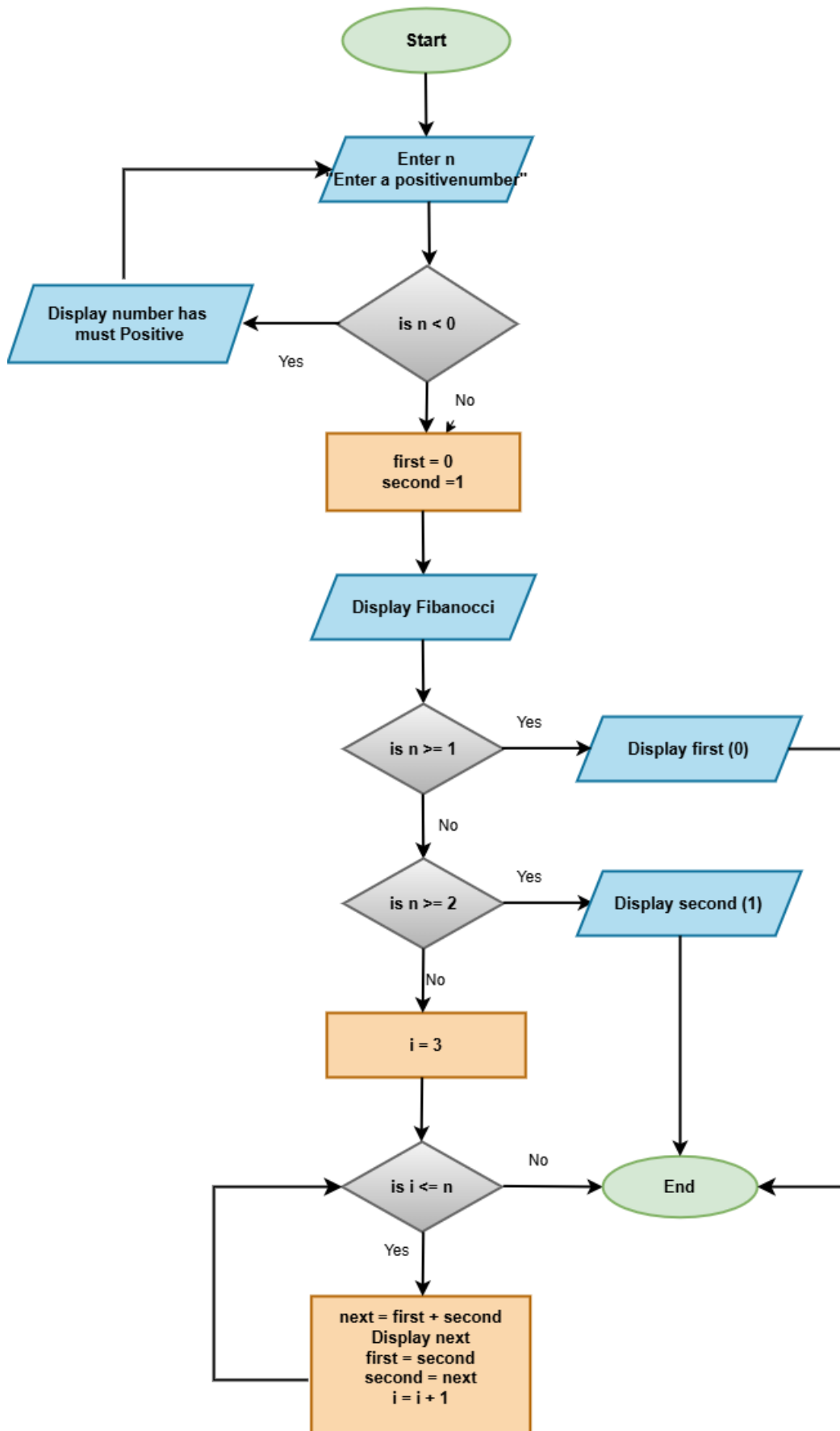
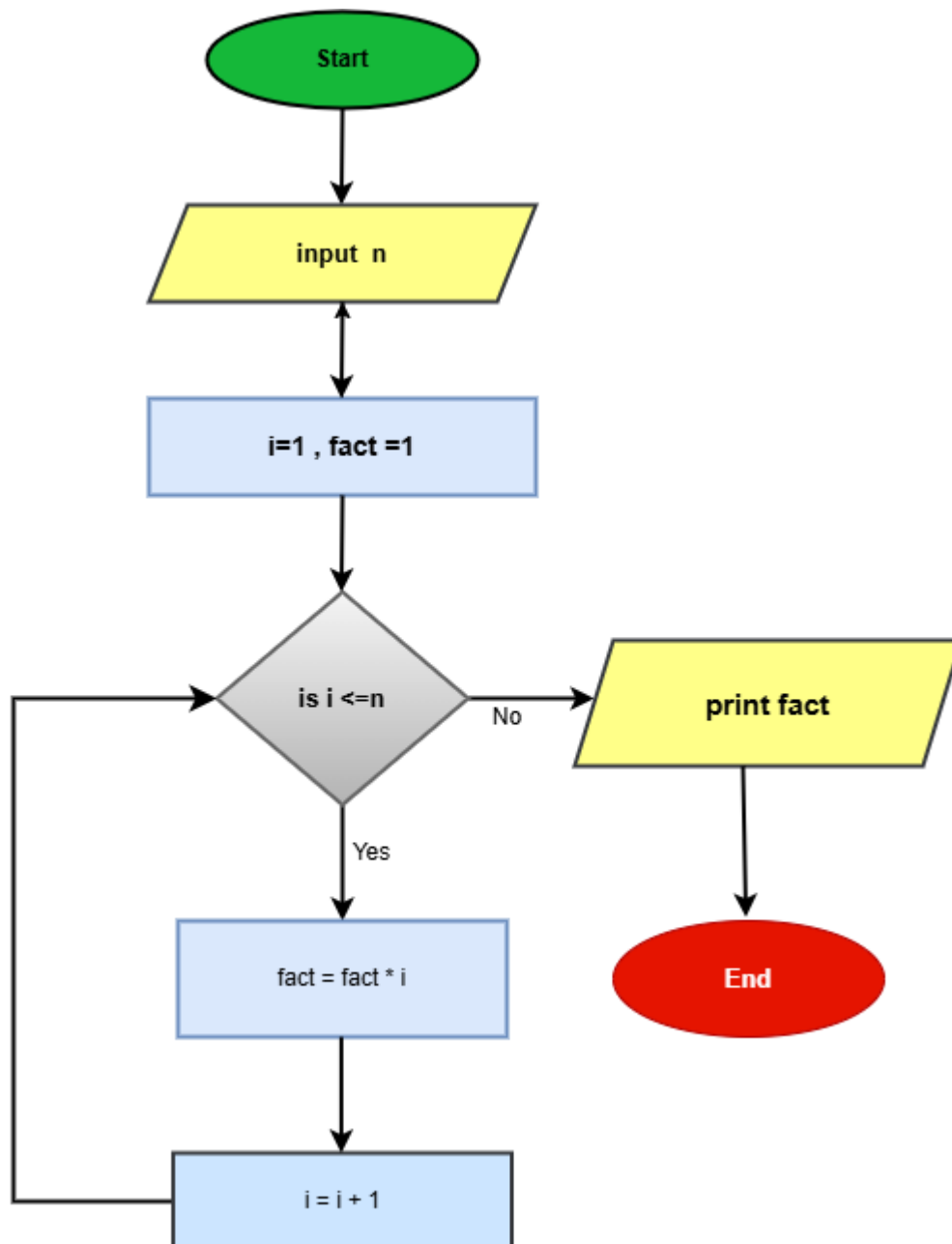


Fibanocci and Factorial Assigment(202507923)

Fibanocci workflow



Factorial Workflow



Factorial Psuedo code

FUNCTION factorial_recursive(n)

IF n == 0 THEN

RETURN 1 // Base case: $0! = 1$

ELSE

RETURN n * factorial_recursive(n - 1) // Recursive call using the formula

END IF

END FUNCTION

MAIN PROGRAM

PRINT program header and formula description

REPEAT

PROMPT user for a number

READ input

IF input is not an integer THEN

DISPLAY error message

ELSE IF input < 0 THEN

DISPLAY error message

ELSE

EXIT loop

END IF

END REPEAT

result $\leftarrow$ factorial_recursive(number)

PRINT number, "!" = ", result

END MAIN PROGRAM

Fibonacci Psuedo code

FUNCTION fibonacci_recursive(n)

 // F(0) = 0

 // F(1) = 1

 // F(n) = F(n-1) + F(n-2) for n >= 2

IF n == 0 THEN

 RETURN 0 // First base case

ELSE IF n == 1 THEN

 RETURN 1 // Second base case

ELSE

 RETURN fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2) // Recursive formula

END IF

END FUNCTION

MAIN PROGRAM

PRINT program header and instructions

REPEAT

 PROMPT user for n

 READ input

 IF input is not an integer THEN

 DISPLAY error message

 ELSE IF n < 0 THEN

 DISPLAY error message

 ELSE

 EXIT loop

 END IF

END REPEAT

result ← fibonacci_recursive(n)

PRINT "F(", n, ") = ", result

END MAIN PROGRAM

Python Implementation(Factorial)

```
def factorial_recursive(n: int) -> int:
    # Base case: factorial of 0 or 1 is 1
    if n == 0 or n == 1:
        return 1
    # Recursive case: n! = n * (n-1)!
    else:
        return n * factorial_recursive(n - 1)

def main() -> None:
    """
    Main driver function. Handles user interaction and displays the result.
    """
    print("          RECURSIVE FACTORIAL CALCULATOR")

    # Input validation loop
    while True:
        try:
            user_input = input("Enter a non-negative integer: ")
            number = int(user_input)

            if number < 0:
                print("Error: Factorial is not defined for negative numbers.")
                continue
            break
        except ValueError:
            print("Error: Invalid input. Please enter a valid integer.")

    # Compute factorial using the recursive function
    result = factorial_recursive(number)

    # Display the result clearly
    print("\n" + "-" * 40)
    print(f"    {number}! = {result}")
    print("-" * 40)

    # Additional explanation of the calculation (optional)
    if number > 1:
        print(f"\nRecursive expansion: {number}! = {number} * {number-1}!")

if __name__ == "__main__":
    main()
```

Python Implentation(Fibanocci)

```
"""Function defines the formula to use when finding the nth Fibonacci number. The main function
handles user input and displays the result."""
def fibonacci_recursive(n: int) -> int:
    """Returns the nth Fibonacci number using recursion."""
    # Base case: F(0) = 0
    if n == 0:
        return 0
    # Base case: F(1) = 1
    elif n == 1:
        return 1
    # Recursive case: apply the formula  $F(n) = F(n-1) + F(n-2)$ 
    else:
        return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

def main() -> None:
    """
    Main driver function. Handles user interaction, input validation,
    and displays the computed Fibonacci number.
    """

    print("      RECURSIVE FIBONACCI NUMBER CALCULATOR")
    print("The Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...")
    print("Enter a position n to get the nth Fibonacci number.\n")

    # Input validation: ensure a non-negative integer is provided
    while True:
        try:
            user_input = input("Enter a non-negative integer n: ")
            n = int(user_input)

            if n < 0:
                print("Error: Please enter a non-negative integer (n >= 0).")
                continue
            break
        except ValueError:
            print("Error: Invalid input. Please enter a valid integer.")

    # Compute the Fibonacci number using the recursive function
    result = fibonacci_recursive(n)

    # Display the result clearly
    print("\n" + "-" * 40)
    print(f"    F({n}) = {result}")
    print("-" * 40)

    # Optional: show the explicit formula application for n > 1
    if n >= 2:
        print(f"\n[Formula applied] F({n}) = F({n-1}) + F({n-2})")

if __name__ == "__main__":
    main()
```